BP101T COMPANION TEXT

Python for Pharmacists

Unit IV · Data Handling with Pandas

BioDockify

Building the Future of AI-Driven Pharmaceutical Research

learn.biodockify.com

Course BP101T · B.Pharm Semester I

Chapters 13–14 · Six teaching hours

Theory-first · Series & DataFrames, CSV & Excel, cleaning, filtering, groupby · ADR case study

Basics of Python Programming for Pharmaceutical Sciences

Contents

Chapter 13 — Pandas Fundamentals....1

13.1 What Pandas is and why it exists....1

13.2 The Series — one labelled column....2

13.3 The DataFrame — a full table....3

13.4 Reading CSV and Excel files (syllabus point)....5

13.5 Inspecting datasets — head, tail, info, describe (syllabus point)....5

Chapter 14 — Cleaning, Filtering and Grouping....7

14.1 Missing values — the NaN problem (syllabus point)....7

14.2 Filtering and selecting data (syllabus point)....8

14.3 Grouping and aggregation (syllabus point)....10

14.4 The ADR case study — the complete workflow....11

Unit IV — Consolidation....12

Syllabus coverage map....12

Twenty-question exercise bank....12

Answers to selected exercises....15

Important points for examination....15

Right-click the table of contents and choose "Update Field" to refresh page numbers.

Unit hours: 6 · **Book chapters:** 13–14 · **Syllabus bullets covered:** introduction to the Pandas library; Pandas Series and DataFrame structures; reading CSV and Excel files — PK study datasets and ADR reports; inspecting datasets using head(), tail(), info() and describe(); data cleaning techniques and handling missing values; filtering and selecting data based on conditions; grouping data and performing aggregation functions.

Unit objectives. After studying this unit you should be able to: (1) explain what Pandas adds over plain Python; (2) create and manipulate Series and DataFrames; (3) load CSV and Excel files into DataFrames; (4) inspect a dataset's structure, size and quality; (5) clean missing values, wrong types and duplicates; (6) filter and select data by conditions; and (7) group and aggregate data to produce summary tables.

Chapter 13 — Pandas Fundamentals

13.1 What Pandas is and why it exists

Pandas is Python's open-source package for tabular data — the addition that made the language a serious analytical platform rather than just a scripting tool. Sitting above NumPy (Unit III), it supplies fast, flexible containers purpose-built for the spreadsheet-shaped data pharmacists work with: PK study sheets, ADR reports, hospital records, clinical trial exports. Install with `pip install pandas`, import with the universal alias:

```
import pandas as pd
```

What Pandas adds over the raw Python of Unit III:

- **Direct file access** — one function reads an entire CSV or Excel file into a ready-to-analyse table.
- **Missing-data handling** — clinical datasets arrive with gaps; Pandas represents each as `NaN` (Not a Number) and provides clean removal or replacement.
- **Fast filtering** — "all patients with Cmax above 50" is one line, not a loop.
- **Grouping and statistics** — average dose per drug category, count of ADRs per severity band, in a single expression.
- **Integration** — Pandas works seamlessly with NumPy (Unit III) and Matplotlib (Unit V).

13.2 The Series — one labelled column

A **Series** behaves as a single indexed column: every value carries a readable label, so you address items by name rather than by position.

```
import pandas as pd
doses = pd.Series([500, 850, 1000], index=["Aspirin", "Dolo", "Meftal"])
print(doses)
```

OUTPUT

```
Aspirin     500
Dolo        850
Meftal     1000
dtype: int64
```

Three things to notice: the index labels appear on the left; the values on the right; and the `dtype` line reports the data type of the column.

Access by label, or by position with `.iloc`:

```
print(doses["Dolo"])
print(doses.mean())
print(doses.sum())
```

OUTPUT

```
850
783.3333333333334
2350
```

A Series carries its own statistics — `.mean()`, `.sum()`, `.max()`, `.min()`, `.std()` — so a single column of numbers is a complete analytical object.

13.3 The DataFrame — a full table

A **DataFrame** is Pandas' flagship structure: a labelled grid in which both rows and columns carry names, built from multiple Series aligned over a common index. Every real dataset in this unit and Unit V lives inside one.

Term	Meaning
Column	one attribute or field (e.g. `Age`)
Row / record	one observation (one patient, one ADR)
Index	row labels (default numbering: 0, 1, 2, …)

Build one from a dictionary — keys become column names, values become the columns:

```
data = {
    "Patient_ID": ["P101", "P102", "P103", "P104"],
    "Age": [34, 58, 45, 28],
    "Cmax": [45.2, 38.6, 50.1, 42.8],
    "Tmax": [2.1, 1.8, 2.4, 1.9],
}
df = pd.DataFrame(data)
print(df)
```

```
OUTPUT
Patient_ID  Age  Cmax  Tmax
0       P101   34  45.2   2.1
1       P102   58  38.6   1.8
2       P103   45  50.1   2.4
3       P104   28  42.8   1.9
```

Four rows, four columns, each column a Series, the index running 0–3. Every real dataset you load in this unit and Unit V arrives as a DataFrame exactly like this.

Five examples of selecting from a DataFrame:

```
print(df["Age"])            # one column -> a Series
```

```
OUTPUT
0    34
1    58
2    45
3    28
Name: Age, dtype: int64
```

```
print(df[["Age", "Cmax"]])  # multiple columns -> a DataFrame
```

```
OUTPUT
   Age  Cmax
0   34  45.2
1   58  38.6
2   45  50.1
3   28  42.8
```

```
print(df.columns)           # column names
```

```
OUTPUT
Index(['Patient_ID', 'Age', 'Cmax', 'Tmax'], dtype='object')
```

```
print(df.shape)             # (rows, columns)
```

```
OUTPUT
(4, 4)
```

```
print(df["Cmax"].mean())    # statistics on one column
```

```
OUTPUT
44.175
```

13.4 Reading CSV and Excel files (syllabus point)

The whole point of Pandas is loading real data. Two functions cover the syllabus file formats:

```
pk_data = pd.read_csv("pk_study.csv")
adr_data = pd.read_excel("adr_reports.xlsx", sheet_name="Sheet1")
```

Function	File type	Key parameters
`pd.read_csv()`	`.csv`	`filepath`, `sep=","`, `header=0`
`pd.read_excel()`	`.xlsx` / `.xls`	`filepath`, `sheet_name`
`df.to_csv()`	saves as CSV	`filepath`, `index=False`
`df.to_excel()`	saves as Excel	`filepath`, `index=False`

Note: `pd.read_excel()` for `.xlsx` files requires the `openpyxl` library (`pip install openpyxl`). A `ModuleNotFoundError` on first use usually means this is missing.

Pass `index=False` when saving to avoid writing the row numbers as an extra column.

13.5 Inspecting datasets — head, tail, info, describe (syllabus point)

Before analysing any dataset, inspect its structure, size and quality. Pandas provides four functions for exactly this:

Function	What it shows
`df.head(n)`	the first n rows (default 5)
`df.tail(n)`	the last n rows (default 5)
`df.info()`	column names, data types, non-null counts, memory
`df.describe()`	summary statistics: count, mean, std, min, 25%, 50%, 75%, max

Five inspection examples on a PK study dataset:

```
pk_data = pd.DataFrame({
    "Patient_ID": ["P101", "P102", "P103", "P104", "P105", "P106"],
    "Age": [34, 58, 45, 28, 62, 51],
    "Cmax": [45.2, 38.6, 50.1, 42.8, 47.5, 39.2],
    "Drug": ["Metformin", "Metformin", "Amlodipine", "Metformin", "Amlodipine",
"Metformin"],
})
print(pk_data.head(3))
```

OUTPUT

```
Patient_ID  Age  Cmax        Drug
0       P101   34  45.2   Metformin
1       P102   58  38.6   Metformin
2       P103   45  50.1  Amlodipine
```

```
print(pk_data.tail(2))
```

```
OUTPUT
  Patient_ID  Age  Cmax        Drug
4       P105   62  47.5  Amlodipine
5       P106   51  39.2   Metformin
```

```
print(pk_data.shape)
```

```
OUTPUT
(6, 4)
```

```
print(pk_data["Cmax"].describe())
```

```
OUTPUT
count     6.000000
mean     43.900000
std       4.538744
min      38.600000
25%      40.100000
50%      44.000000
75%      46.925000
max      50.100000
Name: Cmax, dtype: float64
```

`describe()` is the fastest quality check in the toolkit: count reveals missing values, min/max reveal outliers, and the quartiles sketch the distribution — all in one call.

Chapter summary. Pandas (import as `pd`) delivers two structures: the **Series** (a labelled one-dimensional column with built-in statistics) and the **DataFrame** (a labelled two-dimensional table — the working object for real datasets). `read_csv()` and `read_excel()` load files directly; `to_csv()` and `to_excel()` with `index=False` save them back. Inspection is a ritual: `head()`/`tail()` to see rows, `.shape` for size, `columns` for names, `info()` for types and gaps, `describe()` for statistics. The workflow that drives Unit IV: **Read → Inspect → Clean → Filter → Group**.

Quick checks 13. (a) Series vs DataFrame in one sentence each. (b) What does `df.shape` return for 120 rows and 6 columns? (c) Which function gives you mean, min and max in one call? (d) What extra library does `read_excel()` need for `.xlsx`? (e) Why pass `index=False` to `to_csv()`?

Chapter 14 — Cleaning, Filtering and Grouping

14.1 Missing values — the NaN problem (syllabus point)

Real PK/ADR datasets arrive with holes. Pandas marks each missing entry as **NaN** (Not a Number), and the first cleaning job is to find and handle them.

Function	Purpose
`df.isnull()`	True/False for every cell — is it missing?
`df.isnull().sum()`	count of missing values per column
`df.dropna()`	removes rows (or columns) containing NaN
`df.fillna(value)`	replaces NaN with a value (0, mean, etc.)
`df.drop_duplicates()`	removes duplicate rows
`df.astype(type)`	converts a column to a different type

Five cleaning examples:

```
pk = pd.DataFrame({
    "Age": [34, None, 45, 28, None],
    "Cmax": [45.2, 38.6, None, 42.8, 47.5],
})
print(pk.isnull().sum())
```

OUTPUT

```
Age     2
Cmax    1
dtype: int64
```

```
# Fill missing ages with the column's mean
pk["Age"] = pk["Age"].fillna(pk["Age"].mean())
print(pk["Age"])
```

OUTPUT

```
0    34.0
1    35.666666666666664
2    45.0
3    28.0
4    35.666666666666664
Name: Age, dtype: float64
```

```
# Drop rows where Cmax is missing
pk_clean = pk.dropna(subset=["Cmax"])
print(pk_clean.shape)
```

OUTPUT

```
(4, 2)
```

```
# Remove duplicates
pk = pk.drop_duplicates()
print(pk.shape)
```

OUTPUT

```
(4, 2)
```

```
# Convert a text column to numeric
df = pd.DataFrame({"Dose_mg": ["500", "250", "750"]})
df["Dose_mg"] = df["Dose_mg"].astype(int)
print(df["Dose_mg"].mean())
```

OUTPUT

```
500.0
```

The cleaning decision — fill with the mean, drop the row, or something else — is clinical, not computational. Filling an age with the average is convenient; filling a creatinine with the average could be dangerous. The pharmacist decides; Pandas executes.

14.2 Filtering and selecting data (syllabus point)

Filtering answers screening questions: *which patients are above 60? which readings exceed a threshold?* The syntax is one line: square brackets containing a condition on a column.

Selecting columns:

```
pk_data = pd.DataFrame({
    "Patient_ID": ["P101", "P102", "P103", "P104"],
    "Age": [34, 58, 45, 28],
    "Cmax": [45.2, 38.6, 50.1, 42.8],
    "Drug": ["Metformin", "Metformin", "Amlodipine", "Metformin"],
})
print(pk_data["Cmax"])              # single column -> Series
```

OUTPUT

```
0    45.2
1    38.6
2    50.1
3    42.8
Name: Cmax, dtype: float64
```

Filtering rows by condition:

```
high_cmax = pk_data[pk_data["Cmax"] > 45]
print(high_cmax)
```

OUTPUT

```
Patient_ID  Age  Cmax        Drug
0       P101   34  45.2   Metformin
2       P103   45  50.1  Amlodipine
```

The condition `pk_data["Cmax"] > 45` produces a column of True/False; enclosing it in `pk_data[...]` keeps only the True rows.

More filter examples:

```
elderly = pk_data[pk_data["Age"] >= 45]
print(len(elderly))
```

OUTPUT

```
2
```

```
# Multiple conditions: & (and), | (or) — each condition in brackets
result = pk_data[(pk_data["Age"] >= 30) & (pk_data["Cmax"] > 40)]
print(result)
```

OUTPUT

```
Patient_ID  Age  Cmax        Drug
0       P101   34  45.2   Metformin
2       P103   45  50.1  Amlodipine
```

```
on_metformin = pk_data[pk_data["Drug"] == "Metformin"]
print(len(on_metformin))
```

OUTPUT

```
3
```

```
# loc[] and iloc[] — label-based and position-based selection
print(pk_data.loc[0, "Age"])        # row label 0, column "Age"
print(pk_data.iloc[2, 2])           # integer position row 2, column 2
```

OUTPUT

```
34
50.1
```

Method	Selects by	Example
`df.loc[label, col]`	label or condition	`df.loc[0, "Age"]`
`df.iloc[i, j]`	integer position	`df.iloc[2, 2]`

14.3 Grouping and aggregation (syllabus point)

`groupby()` partitions rows according to the unique values found in a chosen column; whatever aggregation follows (`.mean()`, `.count()`, `.sum()`, `.agg([...])`) then condenses each partition into a single summary line. This is the everyday tool for ADR analysis by drug, or PK parameters by age band.

Group → count:

```
adr = pd.DataFrame({
    "Drug_Name": ["Metformin", "Amoxicillin", "Ibuprofen", "Metformin",
"Ibuprofen", "Metformin"],
    "Severity": ["Mild", "Moderate", "Serious", "Mild", "Mild", "Moderate"],
    "Age": [58, 34, 62, 45, 28, 51],
})
counts = adr.groupby("Drug_Name")["Severity"].count()
print(counts)
```

OUTPUT

```
Drug_Name
Amoxicillin     1
Ibuprofen       2
Metformin       3
Name: Severity, dtype: int64
```

Group → mean:

```
pk = pd.DataFrame({
    "Age_Group": ["Adult", "Adult", "Elderly", "Elderly"],
    "Cmax": [45.2, 38.6, 50.1, 47.5],
})
avg = pk.groupby("Age_Group")["Cmax"].mean()
print(avg)
```

OUTPUT

```
Age_Group
Adult      41.9
Elderly    48.8
Name: Cmax, dtype: float64
```

More aggregation examples:

```
# Multiple aggregation functions at once
summary = pk.groupby("Age_Group")["Cmax"].agg(["mean", "max"])
print(summary)
```

OUTPUT

```
           mean   max
Age_Group
Adult      41.9  45.2
Elderly    48.8  50.1
```

```
# Sort a groupby result by count, descending
top = counts.sort_values(ascending=False)
print(top)
```

OUTPUT

```
Drug_Name
Metformin       3
Ibuprofen       2
Amoxicillin     1
Name: Severity, dtype: int64
```

```
# Average age per severity band
avg_age = adr.groupby("Severity")["Age"].mean()
print(avg_age)
```

OUTPUT

```
Severity
Moderate    42.5
Mild        43.666666666666664
Serious     62.0
Name: Age, dtype: float64
```

14.4 The ADR case study — the complete workflow

The syllabus names PK study datasets and ADR reports as the target data. Here is a small but complete ADR analysis that exercises every skill in this unit:

```
import pandas as pd

adr = pd.DataFrame({
    "Drug_Name": ["Metformin", "Amoxicillin", "Ibuprofen", "Metformin",
                  "Ibuprofen", "Metformin", "Amlodipine", "Amoxicillin"],
    "Severity": ["Mild", "Moderate", "Serious", "Mild",
                 "Mild", "Moderate", "Serious", "Mild"],
    "Age": [58, 34, 62, 45, 28, 51, None, 41],
})

# 1. Inspect
print(adr.shape)
print(adr.isnull().sum())

# 2. Clean — fill the missing age with the mean
adr["Age"] = adr["Age"].fillna(adr["Age"].mean())

# 3. Filter — serious cases
serious = adr[adr["Severity"] == "Serious"]
print(f"Serious ADR cases: {len(serious)}")

# 4. Group — count per drug, sorted
summary = adr.groupby("Drug_Name")["Severity"].count()
print(summary.sort_values(ascending=False))
```

```
OUTPUT
(8, 3)
Drug_Name    0
Severity     0
Age          1
dtype: int64
Serious ADR cases: 2
Drug_Name
Metformin      3
Amoxicillin    2
Ibuprofen      2
Amlodipine     1
Name: Severity, dtype: int64
```

Every line is a unit skill: `shape` and `isnull().sum()` inspect; `fillna` cleans; a condition filters; `groupby().count()` + `sort_values()` aggregates and ranks. Change the dataset, re-run, and the report updates — this is the shape of Unit V's plots and of every real pharmacovigilance query.

Chapter summary. `isnull().sum()` counts gaps per column; `dropna(subset=[...])` removes incomplete rows; `fillna(mean)` patches numeric gaps — the clinical choice of which is the pharmacist's. Filter with `df[df["col"] > x]`; join conditions via `&`/`|` each inside brackets; address by label with `loc[]`, by position with `iloc[]`. `groupby("col")` splits the data; `.count()`/`.mean()`/`.sum()`/`.agg([f1, f2])` summarise each group; `sort_values(ascending=False)` ranks. The five-step workflow — Read → Inspect → Clean → Filter → Group — is the unit's contribution to every analysis that follows.

Quick checks 14. (a) What does `df.isnull().sum()` tell you? (b) `df[df["Age"] > 60]` — what happens? (c) Write one line: average Cmax per Drug group. (d) `loc` vs `iloc` in one sentence. (e) Why is filling a missing creatinine with the column mean a clinical decision?

Unit IV — Consolidation

Syllabus coverage map

Syllabus bullet	Where
Introduction to the Pandas library	Ch 13 (§13.1)
Pandas Series and DataFrame structures	Ch 13 (§13.2–13.3)
Reading CSV and Excel files — PK datasets and ADR reports	Ch 13 (§13.4)
Inspecting datasets: head(), tail(), info(), describe()	Ch 13 (§13.5)
Data cleaning and handling missing values	Ch 14 (§14.1)
Filtering and selecting data based on conditions	Ch 14 (§14.2)
Grouping data and performing aggregation functions	Ch 14 (§14.3–14.4)

Twenty-question exercise bank

Predict the output (1–8).

1. `df.shape` for 120 rows, 6 columns?
2. `print(pd.Series([10, 20, 30]).mean())`
3. `df = pd.DataFrame({"A": [1, 2, 3]}); print(df["A"].sum())`
4. `df.head(2)` on a 5-row DataFrame — how many rows appear?
5. `df[df["Age"] > 50]` on ages [34, 58, 45, 62] — how many rows?
6. `df.isnull().sum()` on a column with 2 NaN values out of 10?
7. `df.groupby("Drug")["Dose"].mean()` — what does it produce?
8. `df.iloc[0, 1]` — what does it select?

Short answers (9–14).

9. What is the difference between a Series and a DataFrame?
10. Name the four inspection functions and what each shows.
11. `dropna()` vs `fillna()` — when would you use each?
12. What does `&` do in a Pandas filter? Why are brackets needed?
13. What does `groupby()` actually do to the data, conceptually?
14. What is the five-step Pandas workflow?

Write programs (15–20).

15. Create a 4-row DataFrame with Patient_ID, Age, Drug columns; print `head()`.

16. Load `pk_study.csv`, print `.info()` and `.describe()`.
17. Fill missing Cmax values with the column median.
18. Filter patients above 50 on Metformin.
19. Group an ADR dataset by Severity and count each group.
20. Group a PK dataset by Drug; compute mean and max Cmax in one `.agg()` call.

Answers to selected exercises

1. `(120, 6)`. 2. `20.0`. 3. `6`. 4. 2. 5. 2 (ages 58 and 62). 6. `2` for that column. 7. A Series mapping each drug name to its average dose. 8. Row position 0, column position 1. 10. head/tail show rows; info shows column types and non-null counts; describe gives count/mean/std/min/quartiles/max. 11. dropna removes incomplete rows (when the record is unusable); fillna replaces NaN (when the gap is tolerable). 12. Element-wise AND; brackets make each comparison evaluate before the `&` combines them. 13. It splits rows into groups by the column's unique values, ready for an aggregation function to summarise each group.

Important points for examination

1. Pandas is an external package layered on NumPy, designed for tabular data; the universal import line is `import pandas as pd`.
2. A **Series** holds one indexed column of values; a **DataFrame** is a grid of labelled rows and columns — the working table for every real dataset.
3. `pd.read_csv("file.csv")` loads a CSV; `pd.read_excel("file.xlsx")` loads an Excel sheet (needs `openpyxl`); `df.to_csv("file", index=False)` saves without the row-number column.
4. The four inspection functions: `head(n)`/`tail(n)` show first/last rows; `info()` gives column names, dtypes and non-null counts; `describe()` gives count, mean, std, min, 25%, 50%, 75%, max; `shape` gives `(rows, columns)`.
5. `isnull().sum()` counts missing values (NaN) per column; `dropna()` removes rows with NaN; `fillna(value)` replaces NaN — the choice between dropping and filling is a clinical decision.
6. `drop_duplicates()` removes repeated rows; `astype(type)` converts a column (e.g. text `"500"` to integer 500).
7. Filtering: `df[df["col"] > x]` keeps rows satisfying the condition; multiple conditions use `&` (and) and `|` (or), each comparison wrapped in brackets.
8. `loc[]` addresses rows/columns by their labels; `iloc[]` addresses them by numeric position.
9. `groupby("column")` partitions rows by the column's unique values; `.count()`, `.mean()`, `.sum()`, `.median()` or `.agg(["mean", "max"])` then condense each partition.

10. The Pandas workflow: **Read → Inspect → Clean → Filter/Select → Group & Aggregate** — every analysis in this unit and Unit V follows this shape.

Sources for this unit: course materials; topic coverage cross-checked against Noteskarts *Computer Applications in Pharmacy — Unit 4* notes, pandas official documentation (pandas.pydata.org), MCA R24 lecture notes, and the curated learn.biodockify.com catalog (pandas-dev/pandas, numpy, exercism/python, W3Schools). All prose and examples are original; code outputs verified by execution; originality audited against every consulted source. Full credits: see the book's Sources and Acknowledgements page.